

Python

Словари, frozenset



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение



Множество



Хеш



Методы добавления и удаления элементов



Операции над множествами

План занятия

- Словари, frozenset
- frozenset
- Словарь
- Преобразование в словарь
- Оператор in
- Цикл по словарю
- Добавление и обновление данных
- Удаление данных



ОСНОВНОЙ БЛОК





Словари, **frozenset**



Set comprehension

Это способ создания множества на основе другой последовательности или итерируемого объекта с применением условий и выражений. Синтаксис полностью аналогичен `List comprehension`, за исключением фигурных скобок вместо квадратных.

Синтаксис



```
new_set = {expression for item in iterable}
```

Пример



```
# Создание множества с числами
numbers = [1, 2, 2, 4, 7, 8, 8, 10]
even_numbers_set = {num for num in numbers if num % 2 == 0}
print(even_numbers_set)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
words = ["apple", "banana", "cherry",
         "apple"]
unique_lengths = {len(word) for word in
                  words}
print(unique_lengths)
```

- a. {5, 6}
- b. [5, 6]
- c. {4}
- d. {5, 6, 6, 5}



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
words = ["apple", "banana", "cherry",
         "apple"]
unique_lengths = {len(word) for word in
                  words}
print(unique_lengths)
```

- a. {5, 6}
- b. [5, 6]
- c. {4}
- d. {5, 6, 6, 5}



ВОПРОСЫ





frozenset



frozenset

Это неизменяемый аналог обычного множества. В отличие от множества, элементы **frozenset** не могут быть добавлены, удалены или изменены после создания.

frozenset



`frozenset` создаётся с помощью одноимённой функции, в которую передается итерируемый объект.

frozenset пример



```
immutable_set = frozenset([1, 2, 3, 4, 5])  
  
print(immutable_set)  
  
immutable_from_range = frozenset(range(10))  
  
print(immutable_from_range)
```

Хешируемость frozenset



`frozenset` является неизменяемым и хешируемым объектом. Это позволяет использовать его в качестве элемента множества, в отличие от обычного множества `set`, которое не может быть элементом другого множества из-за своей изменяемости.

Пример



```
# Создание frozenset
```

```
frozen_set1 = frozenset([1, 2, 3])
```

```
frozen_set2 = frozenset([4, 5, 6])
```

```
# Создание множества, содержащего frozenset
```

```
set_of_frozensets = {frozen_set1, frozen_set2}
```

```
print(set_of_frozensets)
```

Сравнение set и frozenset



Это оба типа множеств, которые поддерживают уникальные элементы и позволяют выполнять множество операций, таких как объединение, пересечение и разность. Однако между ними есть важные различия, которые влияют на их применение.

Характеристика	set	frozenset
Изменяемость	Изменяемый: поддерживает методы <code>add()</code> , <code>remove()</code> , <code>discard()</code> , <code>pop()</code> , <code>clear()</code> .	Неизменяемый: методы изменения отсутствуют. Поддерживаются только методы, возвращающие новый объект.
Хешируемость	Не является хешируемым, поэтому не может быть элементом других множеств или ключом словаря.	Хешируемый: может использоваться как элемент множества или ключ словаря.



ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

Что из перечисленного верно для `frozenset`?

- a. Это изменяемое множество, поддерживающее добавление элементов.
- b. Это неизменяемое множество, элементы которого нельзя изменять после создания.
- c. Это хешируемый тип данных.
- d. `frozenset` поддерживает методы `add()` и `remove()`.



Выберите верный вариант ответа

Что из перечисленного верно для `frozenset`?

- a. Это изменяемое множество, поддерживающее добавление элементов.
- b. Это неизменяемое множество, элементы которого нельзя изменять после создания.
- c. Это хешируемый тип данных.
- d. `frozenset` поддерживает методы `add()` и `remove()`.



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
immutable_set = frozenset([1, 2, 3])
new_set = immutable_set.union({4, 5})
print(new_set)
```

- a. `frozenset({1, 2, 3, 4, 5})`
- b. `{1, 2, 3, 4, 5}`
- c. `{4, 5}`
- d. Ошибка



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
immutable_set = frozenset([1, 2, 3])
new_set = immutable_set.union({4, 5})
print(new_set)
```

- a. `frozenset({1, 2, 3, 4, 5})`
- b. `{1, 2, 3, 4, 5}`
- c. `{4, 5}`
- d. Ошибка



ВОПРОСЫ





Словарь



Словарь (ассоциативный массив)

Это структура данных, представляющая собой неупорядоченную изменяемую коллекцию пар "ключ-значение"

1. Создание словаря с использованием фигурных скобок



Синтаксис

```
my_dict = {  
    key1: value1,  
    key2: value2,  
    ...  
}
```

Пояснение

- **key** — уникальный объект (ключ), по которому можно получить соответствующее значение.
- **value** — данные (значение), которые хранятся по ключу.

Создание словаря с использованием фигурных скобок



Пример

```
person = {"name": "Alice", "age": 30,  
          "city": "New York"}  
  
print(person)
```

Пояснение

Наиболее распространённый способ создания словаря, в котором указываются ключи и соответствующие им значения

Создание пустого словаря



Пример

```
empty_dict = {}  
print(empty_dict)  
  
empty_dict = dict()  
print(empty_dict)
```

Пояснение

Создать пустой словарь для последующего добавления элементов можно с помощью фигурных скобок или функции `dict()`.

Хранение словаря в памяти

Хеширование ключей

Хеш-функция вычисляет уникальное хеш-значение, что позволяет словарям быстро находить значение по ключу

Хеш-таблица

Хранение пар "ключ-значение". При одинаковом хеш-значении (коллизии), используется метод цепочек для разрешения конфликта

Занимаемая память

Размер таблицы превышает размер ее содержимого. При добавлении новых элементов возможно рехеширование

Порядок элементов

Начиная с Python 3.7, словари сохраняют порядок вставки элементов. Использование упорядоченных хеш-таблиц

Хранение словаря в памяти



Преимущества

- Быстрый доступ
- Гибкость
- Уникальные ключи

Ограничения

- Память
- Изменяемость ключей

Особенности словарей



Ключи должны быть уникальными и хешируемыми



Гибкость значений



Быстрый доступ к значениям



Сохранение порядка элементов (начиная с Python 3.7)



Изменяемость значений

Особенности хеширования

Пояснения

В Python числа `1`, `1.0` и `True` обладают одинаковым хешем:

- При сравнении `1 == 1.0 == True` результатом будет `True`.
- Функция `hash` вернет единицу для значений `1`, `1.0` и `True`.

Проверка

```
print(hash(1))
```

```
print(hash(1.0))
```

```
print(hash(True))
```

Последствия для словарей и множеств

При использовании 1, 1.0 и True в качестве ключей в одном словаре или элементах множества, они будут рассматриваться как один и тот же ключ или элемент.

```
my_dict = {1.0: "float", 1: "integer", True: "boolean"}
print(my_dict)
```

В этом примере значение 1.0: "float" будет перезаписано значением 'boolean', так как все три ключа считаются одинаковыми. При этом ключ остается первый из добавленных.

Множества также будут содержать только один из таких элементов:

```
my_set = {True, 1, 1.0}
print(my_set)
```




ВОПРОСЫ





ЗАДАНИЕ

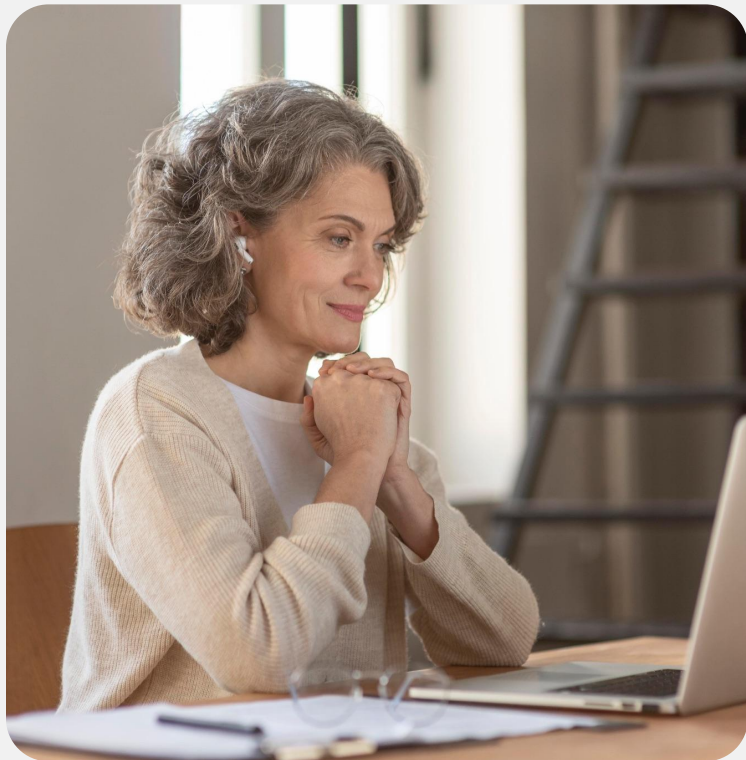




Выберите правильные варианты ответа

Какие из перечисленных типов данных можно использовать в качестве ключа в словаре?

- a. `int`
- b. `bool`
- c. `set`
- d. `float`
- e. `dict`
- f. `tuple`
- g. `list`
- h. `frozenset`



Выберите правильные варианты ответа

Какие из перечисленных типов данных можно использовать в качестве ключа в словаре?

- a. int
- b. bool
- c. set
- d. float
- e. dict
- f. tuple
- g. list
- h. frozenset



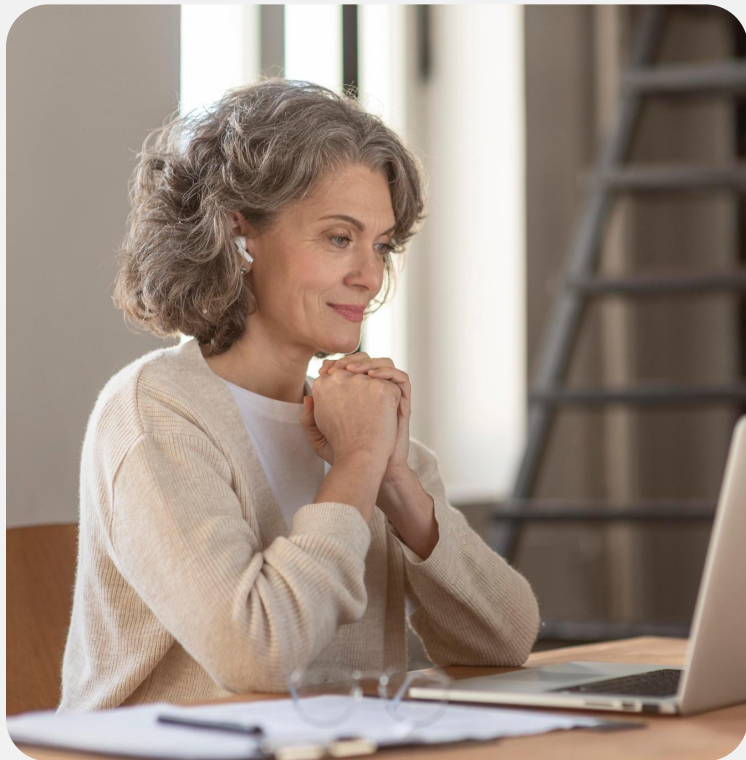
Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {1: "one", 1.0: "float one",
True: "boolean one"}
```

```
print(my_dict)
```

- a. {1: "one", 1.0: "float one", True: "boolean one"}
- b. {1: "one"}
- c. {1: "boolean one"}
- d. {1.0: "float one"}



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {1: "one", 1.0: "float one",
True: "boolean one"}
```

```
print(my_dict)
```

- a. {1: "one", 1.0: "float one", True: "boolean one"}
- b. {1: "one"}
- c. {1: "boolean one"}
- d. {1.0: "float one"}



ВОПРОСЫ





Преобразование в словарь

Особенности преобразования



Ключи в словаре должны быть хешируемыми и уникальными



При преобразовании коллекций с дублирующимися ключами будет сохранено только последнее значение



Если структура данных не совместима с форматом "ключ-значение", преобразование в словарь приведёт к ошибке

Преобразование именованных аргументов в словарь

Пример

```
# Словарь с использованием функции dict()

person = dict(name="Bob", age=25,
               city="London")

print(person)
```

Пояснения

Функция `dict()` позволяет создавать словарь из пар ключ-значение, передаваемых как именованные аргументы

Преобразование других коллекций в словарь



Функция `dict()` может быть использована для преобразования последовательности пар (например, списка кортежей) в словарь. Важно чтобы вложенные коллекции состояли ровно из двух элементов и имели неизменяемый элемент на нулевом индексе

Примеры

Список кортежей

```
pairs = [("name", "Charlie"), ("age", 35), ("city", "Paris")]
person = dict(pairs)
print(person)
```

Список списков

```
pairs = [("name", "Charlie"), ["age", 35], ["city", "Paris"]]
person = dict(pairs)
print(person)
```

Несоответствующее количество элементов

```
not_pairs = [("name", "Charlie"), ["age", 35], ["city", "Paris", "Berlin"]]
person = dict(not_pairs) # Вызовет ошибку
print(person)
```

Хешируемый ключ

```
pairs = ([("name", "surname"], "Charlie"), ["age", 35], ["city", "Paris"])
person = dict(pairs) # Вызовет ошибку
print(person)
```

Доступ к значениям по ключам

Синтаксис

```
variable = dictionary[key]
```

Пояснения

- **dictionary** — объект словаря, из которого вы хотите получить значение
- **key** — ключ, по которому будет извлечено связанное значение
- **variable** — переменная, в которой можно сохранить значение, связанное с указанным ключом

Доступ к значениям по ключам

Особенности

- Если ключ существует, будет возвращено соответствующее значение.
- Если ключ отсутствует, возникает ошибка `KeyError`.

Пример

```
# Получение значения по ключу
my_dict = {"name": "Alice", "age": 30}
print(my_dict["name"])
age = my_dict["age"]
print(age)
# print(my_dict["city"])
# Вызовет ошибку
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
pairs = [("name", "Charlie"), ("age", 35), ("city", "Paris"), ("name", "Bob")]
```

```
person = dict(pairs)
```

```
print(person["name"])
```

- Charlie
- ("name", "Charlie")
- "name": Charlie
- Bob



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
pairs = [("name", "Charlie"), ("age", 35), ("city", "Paris"), ("name", "Bob")]
```

```
person = dict(pairs)
```

```
print(person["name"])
```

- a. Charlie
- b. ("name", "Charlie")
- c. "name": Charlie
- d. Bob



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
not_pairs = [("name", "Charlie"), ("age", 35),
             ["city", "Paris", "Berlin"]]
```

```
person = dict(not_pairs)
```

```
print(person)
```

- `{"name": "Charlie", "age": 35, "city": "Paris"}`
- `{"name": "Charlie", "age": 35, "city": ["Paris", "Berlin"]}`
- `{"name": "Charlie", "age": 35, "city": "Berlin"}`
- Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
not_pairs = [("name", "Charlie"), ["age", 35],
             ["city", "Paris", "Berlin"]]
```

```
person = dict(not_pairs)
```

```
print(person)
```

- `{"name": "Charlie", "age": 35, "city": "Paris"}`
- `{"name": "Charlie", "age": 35, "city": ["Paris", "Berlin"]}`
- `{"name": "Charlie", "age": 35, "city": "Berlin"}`
- Ошибка



ВОПРОСЫ





Оператор in

Оператор in



Используется для проверки, существует ли указанный ключ в словаре. Он часто используется для проверки наличия ключа перед доступом к значению, чтобы избежать ошибки `KeyError`.

Пример



```
my_dict = {"name": "Alice", "age": 30}
if "name" in my_dict:
    print(my_dict["name"]) # Выведет значение по ключу 'name'
if "city" in my_dict:
    print(my_dict["city"]) # Не выполняется, так как ключ 'city' отсутствует
```




ВОПРОСЫ





Цикл по словарю



Цикл по словарю



По словарям можно итерироваться с помощью цикла `for`. По умолчанию цикл `for` перебирает ключи словаря, с помощью которых можно получить значения.

Пример



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}
for key in my_dict:
    print(f"Ключ={key}, значение={my_dict[key]}")
```



ВОПРОСЫ





Добавление и обновление данных

Добавление и обновление данных



Словари позволяют быстро добавлять новые пары "ключ-значение" или обновлять существующие значения по ключу.

Добавление новой пары "ключ-значение"



```
my_dict = {"name": "Alice", "age": 30}  
my_dict["city"] = "New York" # Добавление нового элемента  
print(my_dict)
```


Обновление значения по существующему ключу



```
my_dict = {"name": "Alice", "age": 30}
my_dict["age"] = 31 # Обновление значения по ключу "age"
print(my_dict)
```

Метод update()



Метод `update()` позволяет добавлять несколько пар "ключ-значение" или обновлять значения существующих ключей. Можно передать другой словарь или другую последовательность пар ключ-значение.

Метод update()



Обновление значений и добавление новых ключей

```
my_dict = {"name": "Alice", "age": 30}
my_dict.update({"age": 32, "country": "USA"})
print(my_dict)
```

Обновление с использованием списка кортежей

```
my_dict.update([("name", "Bob"), ("email", "bob@example.com")])
print(my_dict)
```

Обновление с использованием именованных аргументов

```
my_dict.update(city="New York", orders=[])
print(my_dict)
```

Особенности добавления и обновления



- Если ключ уже существует, его значение будет обновлено.
- Если ключ отсутствует, он будет добавлен в словарь.



ВОПРОСЫ





Удаление данных



Оператор del

Этот оператор удаляет элемент с указанным ключом из словаря. Если ключ отсутствует, возникает ошибка `KeyError`.

Оператор del. Пример



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}  
  
del my_dict["age"]  
  
print(my_dict)  
  
# del my_dict["email"] # Вызовет ошибку
```




Метод `clear()`

Этот метод удаляет все элементы из словаря, оставляя пустой словарь.

Метод clear(). Пример



```
my_dict = {"name": "Alice", "age": 30}  
my_dict.clear()  
print(my_dict)
```



Метод pop()

Этот метод удаляет элемент по указанному ключу и возвращает его значение. Если ключ отсутствует и значение по умолчанию не указано, возникает ошибка `KeyError`.

Метод pop(). Пример



```
my_dict = {"name": "Alice", "age": 30}
age = my_dict.pop("age")
print(age)
print(my_dict)

# my_dict.pop("email") # Вызовет ошибку
```



Метод `popitem()`

Удаляет и возвращает последнюю добавленную пару (начиная с Python 3.7) или случайную пару. Если словарь пуст, возникает ошибка `KeyError`.

Метод popitem(). Пример



```
my_dict = {"name": "Alice", "age": 30}

last_item = my_dict.popitem()

print(last_item)

print(my_dict)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
my_dict.update({"city": "New York", "age": 35})
print(my_dict)
```

- a. {"name": "Alice", "age": 30, "city": "New York"}
- b. {"name": "Alice", "age": 35, "city": "New York"}
- c. {"name": "Alice", "age": 30, "age": 35, "city": "New York"}
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
my_dict.update({"city": "New York", "age": 35})
print(my_dict)
```

- a. {"name": "Alice", "age": 30, "city": "New York"}
- b. {"name": "Alice", "age": 35, "city": "New York"}
- c. {"name": "Alice", "age": 30, "age": 35, "city": "New York"}
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
del my_dict["age"]
print(my_dict)
```

- a. {"name": "Alice"}
- b. {"name": "Alice", "age": 30}
- c. {"name": "Alice", "age"}
- d. {"name": "Alice", 30}



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
del my_dict["age"]
print(my_dict)
```

- a. {"name": "Alice"}
- b. {"name": "Alice", "age": 30}
- c. {"name": "Alice", "age"}
- d. {"name": "Alice", 30}



Выберите правильный вариант ответа

Какое значение будет возвращено при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
```

```
value = my_dict.pop("age")
```

```
print(value)
```

- a. {"age": 30}
- b. {"name": "Alice"}
- c. 30
- d. None



Выберите правильный вариант ответа

Какое значение будет возвращено при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
value = my_dict.pop("age")
print(value)
```

- a. {"age": 30}
- b. {"name": "Alice"}
- c. 30
- d. None



Выберите правильный вариант ответа

Какое значение будет возвращено при выполнении следующего кода?

```
my_dict = {"name": "Alice", "age": 30}
value = my_dict.pop("age")
print(value)
```

- a. {"age": 30}
- b. {"name": "Alice"}
- c. 30
- d. None



ВОПРОСЫ





ПРАКТИЧЕСКИЕ ЗАДАНИЯ





1. Инверсия словаря

Напишите программу, которая создаёт новый словарь, где значения из исходного словаря становятся ключами, а ключи — значениями.

Данные:

```
original_dict = {"a": 1, "b": 2, "c": 3}
```

Пример вывода:

Инверсированный словарь: {1: 'a', 2: 'b', 3: 'c'}



2. Из чисел в слова

Напишите программу, которая заменяет числовые значения в словаре на их строковое представление (например, 1 → "один"). Используйте заранее подготовленный словарь чисел.

Данные:

Словарь сопоставлений

```
number_to_word = {1: "один", 2: "два", 3: "три"}
```

Исходные данные

```
data = {"x": 1, "y": 2, "z": 3}
```

Пример вывода:

```
{'x': 'один', 'y': 'два', 'z': 'три'}
```



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

1. Не уникальные числа

Напишите программу, которая находит все числа, встречающиеся более одного раза в списке, и выводит их в порядке убывания.

Данные:

```
numbers = [4, 7, 3, 7, 8, 3, 4, 2, 7, 3, 8, 4]
```

Пример вывода:

Числа, встречающиеся более одного раза: [7, 4, 3, 8]

Домашнее задание

2. Проверка подмножества

Напишите программу, которая проверяет, является ли один словарь подмножеством другого (т.е. все его пары "ключ-значение" содержатся в другом словаре).

Данные:

```
dict1 = {"a": 1, "b": 2}
dict2 = {"a": 1, "b": 2, "c": 3}
```

Пример вывода:

Первый словарь является подмножеством второго.

Заключение

